

# Morphic Studio Software Architecture Specification

Version: 1.0 | Status: Approved

## SUMMARY

---

This document defines the API architecture for Morphic Studio.

## PURPOSE

---

This document defines the API architecture for Morphic Studio.

It establishes how all internal modules, frontend applications, AI services, mobile applications, desktop clients, and future third-party integrations communicate through stable, secure, and versioned interfaces.

The API layer is the official contract between all system components.

## API PHILOSOPHY

---

The API follows nine core principles.

**2.1 API-First Development** Every new feature begins with its API contract before implementation. The API is treated as a product, not merely an implementation detail.

**2.2 Canonical API Contract** All endpoints return a consistent response structure. Every response includes: Success status, Data payload, Metadata, Pagination (where applicable), Trace ID, Timestamp. Every error follows a standardized error contract.

**2.3 Versioned APIs** Expose versioned endpoints (example: /api/v1/). Future breaking changes require new versions. Existing versions remain supported during migration periods.

**2.4 Resource-Oriented Design** Resources represent business concepts: Projects, Characters, Assets, Story Bible, Creative Memory, Worlds, Exports, AI Tasks, Users, Notifications. Avoid action-based endpoint naming where resource-based alternatives exist.

**2.5 Secure by Default** Every endpoint enforces: Authentication, Authorization, Input validation, Rate limiting, Audit logging, Least privilege.

**2.6 Stateless Requests** APIs remain stateless whenever practical. Client state is never relied upon between requests.

**2.7 Event Awareness** APIs may trigger asynchronous domain events without blocking user interactions.

**2.8 AI Independence** No frontend or service communicates directly with AI providers. Every AI request passes through the Unified AI Gateway.

**2.9 Extensibility** The API should support future Public SDKs, Plugins, Marketplace integrations, Enterprise deployments, Mobile applications, Desktop applications without requiring redesign.

## API CATEGORIES

---

The API is organized into capability groups: Authentication, Users, Projects, Story, Story Bible, Characters, Worlds, Assets, Production, Publishing, AI, Notifications, Analytics, Administration.

## REST STANDARDS

---

Use predictable REST conventions. GET: Retrieve resources. POST: Create resources. PUT: Replace resources. PATCH: Partially update resources. DELETE: Remove or archive resources according to business rules. Bulk operations should use dedicated endpoints.

## CANONICAL RESPONSE FORMAT

---

Every successful response should include: success, data, metadata, request\_id, timestamp. Metadata may include: Pagination, Processing time, Warnings, Version, Locale. Every error response should include: Error code, Message, Details (when appropriate), Trace identifier, Suggested recovery action. Never expose sensitive implementation details.

## AUTHENTICATION

---

Support: Session authentication, Secure tokens, Refresh tokens, Future OAuth providers, API keys (future), Multi-device sessions, Session revocation.

## AUTHORIZATION

---

Implement role-based access control. Examples: Owner, Editor, Viewer, Administrator, Future organization roles. Every endpoint validates permissions before execution.

## REAL-TIME COMMUNICATION

---

Use WebSockets for: AI progress, Rendering progress, Collaboration, Presence, Notifications, Autosave synchronization. Realtime communication should degrade gracefully when unavailable.

## AI GATEWAY INTEGRATION

---

All AI-related endpoints communicate through the Unified AI Gateway. Responsibilities include: Provider routing, Prompt assembly, Context injection, Response normalization, Fallback handling, Cost tracking, Rate limiting, Observability. Applications never communicate directly with AI vendors.

## WEBHOOKS

---

Support future outbound events: Project exported, Render completed, Publication succeeded, Asset generated. Webhook requirements: Verification, Retries, Signing, Delivery history, Monitoring.

## SDK STRATEGY

---

Design APIs suitable for official SDKs. Future SDK targets: TypeScript, Python, Swift, Kotlin, C#. SDKs should expose typed interfaces and consistent error handling.

## RATE LIMITING

---

Apply configurable limits. Examples: Authentication, AI generation, Exports, Uploads, Search, Public APIs. Rate limits should balance protection with usability.

## OBSERVABILITY

---

Record: Request latency, Error rates, AI provider usage, API throughput, Authentication failures, Rate-limit violations, Endpoint popularity, Correlation IDs. Support centralized monitoring and distributed tracing.

## EXTERNAL INTEGRATIONS

---

Prepare for future integrations with: Cloud storage, Publishing platforms, Payment providers, Email services, Identity providers, Community platforms, Plugin marketplace. Integrations must use adapter interfaces to avoid vendor lock-in.

## ACCEPTANCE CRITERIA

---

The API architecture is successful if: All system components communicate through stable contracts; Endpoints remain predictable; Clients receive consistent responses; Authentication and authorization are enforced; AI providers remain replaceable; Future integrations require minimal architectural change; Public APIs can be introduced without redesign.

## **CONCLUSION**

---

The Morhic Studio API architecture establishes a secure, versioned, consistent, and extensible communication layer that connects every application, service, and future integration. By enforcing canonical contracts, standardized responses, and centralized AI orchestration, the platform is prepared to evolve while preserving compatibility and developer productivity.